

Exercise series 4: Solutions

Introduction to databases



ADReM
Advanced Database Research & Modelling
University of Antwerp

 **Universiteit
Antwerpen**

Problem 1: Dealers, Producers and Products

Consider the example data provided in the tables Dealers, Producers, Products, and Deals shown below.

- Relax: <https://dbis-uibk.github.io/relax/calc/gist/e0509d68845aaefacf3d36500ed56ccc>
- DB_fiddle: <https://www.db-fiddle.com/f/itidCoYCpBTsgf1jsRPEkd/0>

Dealers	
dealer_id	name
1	Bob's store
2	Appliances Inc.
3	Quality Goods
4	Tech Market

Producers	
producer_id	name
1	United Technics
2	Technics Inc.
3	Big Bang
4	ACME

Products			
product_id	type	price	producer_id
1	vacuum cleaner	66.9	3
2	washing machine	399.99	2
3	tumble dryer	500	2
4	washing machine	780	3
5	oven	175.45	1
6	washing machine	545.9	1
7	vacuum cleaner	475.19	4
8	washing machine	519.99	4
9	tumble dryer	600	2

Deals	
producer_id	dealer_id
1	1
1	2
2	2
2	4
3	1
3	2
3	3
4	2
4	3

You may assume that a dealer, who has a deal with a given producer, sells all the products produced by that producer.



Problem 1: Dealers, Producers and Products

- 1) Give all types of products that are produced by at least two different producers.
- 2) Give the minimum price of a product sold by Quality Goods.
- 3) Give the name of the producer of the most expensive washing machine.
- 4) Give the names of the dealers that do not sell any vacuum cleaners.



1.1 Give all types of products that are produced by at least two different producers.

TP = π type, producer_id (Products)	<code>SELECT DISTINCT P1.type</code>
π type σ producer_id2 $\neq$ producer_id	<code>FROM Products P1, Products P2</code>
$(\rho$ producer_id2 $\leftarrow$ producer_id TP $\bowtie$ TP)	<code>WHERE</code> <code>P1.type = P2.type AND</code> <code>P1.producer_id $\neq$ P2.producer_id;</code>

1.2 Give the minimum price of a product sold by Quality Goods.

Quality_prices =	<code>CREATE VIEW Quality_prices AS</code>	<code>SELECT price</code>
π price	<code>SELECT DISTINCT price</code>	<code>FROM Quality_prices</code>
(	<code>FROM Products</code>	<code>EXCEPT</code>
σ name = 'Quality Goods' Dealers $\bowtie$ Deals	<code>WHERE producer_id IN</code>	<code>SELECT Q1.price</code>
$\bowtie$ Products	(	<code>FROM</code>
)	<code>SELECT producer_id</code>	<code>Quality_prices Q1,</code>
	<code>FROM Deals</code>	<code>Quality_prices Q2</code>
Quality_prices	<code>WHERE dealer_id =</code>	<code>WHERE Q1.price > Q2.price</code>
-	(	
π price σ price > price2	<code>SELECT dealer_id</code>	
(	<code>FROM Dealers</code>	
$\text{Quality_prices} \times$	<code>WHERE name = 'Quality Goods'</code>	
ρ price2 $\leftarrow$ price Quality_prices	)	
)	);	

1.3 Give the name of the producer of the most expensive washing machine.

```
Prod_price =  
π producer_id, price σ type = 'washing machine' Products  
  
Most_expensive =  
π producer_id Prod_price  
-  
π producer_id σ price < price2  
(  
    Prod_price  
    ×  
    ρ price2 ← price, producer_id2 ← producer_id Prod_price  
)  
  
π name (Most_expensive ⋈ Producers)
```

```
CREATE VIEW Prod_price AS  
SELECT producer_id, price  
FROM Products  
WHERE type = 'washing machine';  
  
CREATE VIEW Most_expensive AS  
(  
    SELECT producer_id  
    FROM Prod_price  
    EXCEPT  
    SELECT P1.producer_id  
    FROM Prod_price P1, Prod_price P2  
    WHERE P1.price < P2.price  
);  
  
SELECT name  
FROM Producers  
WHERE producer_id =  
(  
    SELECT *  
    FROM Most_expensive  
);
```

1.4 Give the names of the dealers that do not sell any vacuum cleaners.

```
π name
(
    Dealers
    ⋈
    (
        π dealer_id (Dealers)
        -
        π dealer_id σ type = 'vacuum cleaner'
        (
            Products ⋈ Deals
        )
    )
)
```

```
SELECT name
FROM Dealers
WHERE dealer_id IN
(
    SELECT dealer_id
    FROM Dealers

    EXCEPT

    SELECT dealer_id
    FROM Products NATURAL JOIN Deals
    WHERE type = 'vacuum cleaner'
);
```

Problem 2: Employees and Departments

Consider the example data provided in tables Employees and Departments below.

- Relax: <https://dbis-uibk.github.io/relax/calc/gist/1c947ca70bbc6efffae62eb19d16d023>
- DB_fiddle: <https://www.db-fiddle.com/f/itidCoYCpBTsgf1jsRPEkd/1>

Employees		
name	salary	d_name
John	10000	R & D
John	8000	ICT
Paul	12000	HR
George	8000	ICT
George	7000	R & D
Susan	11000	Fin.
Susan	7000	ICT

Departments	
name	manager
R & D	Bill
HR	Bob
ICT	Bill
PR	Helen
Fin.	Alan
Law	Bob

Note that an employee can be assigned to multiple departments, and a salary is specific to the work done in that department.



Problem 2: Employees and Departments

- 1) Give all departments in which all employees have different salaries.
- 2) Give all pairs of employees that have the same salary in the same department.
Make sure you give every pair just once.
- 3) Give all employees assigned to all departments in which Bill is manager.
- 4) Give all employees assigned to any department in which Bob is manager.



2.1 Give all departments in which all employees have different salaries.

π name Departments

-

π d_name σ name < name2

(

Employees $\bowtie$ ρ name2 $\leftarrow$ name Employees

)

SELECT DISTINCT name

FROM Departments E

WHERE NOT EXISTS

(

SELECT D1.name, D2.name

FROM Employees D1 JOIN Employees D2

ON (D1.d_name, D1.salary) = (D2.d_name, D2.salary)

WHERE

D1.name <> D2.name AND

D1.d_name = E.name

);

2.2 Give all pairs of employees that have the same salary in the same department. Make sure you give every pair just once.

π name, name2 σ name < name2

(

Employees $\bowtie$ ρ name2 $\leftarrow$ name Employees

)

SELECT D1.name, D2.name

FROM Employees D1 JOIN Employees D2

ON (D1.d_name, D1.salary) = (D2.d_name, D2.salary)

WHERE D1.name < D2.name ;

2.3 Give all employees assigned to all departments in which Bill is manager.

π name, d_name Employees
 $\div$
 ρ d_name $\leftarrow$ name π name σ manager = 'Bill' Departments

```
SELECT E.name
FROM Employees E JOIN Departments D ON E.d_name =
D.name
WHERE D.manager = 'Bill'
```

EXCEPT

```
SELECT S.name
FROM
(
    SELECT E.name AS name, D.name as dep
    FROM Employees E, Departments D
    WHERE D.manager = 'Bill'

    EXCEPT

    SELECT E.name, E.d_name
    FROM Employees E JOIN Departments D
    ON E.d_name = D.name
    WHERE D.manager = 'Bill'
) AS S;
```

$A \div B := \pi \text{ unique } A - \pi \text{ unique } (\pi \text{ unique } A \times B - A \bowtie B)$

A	
unique	common
1	'X'
1	'Y'
2	'X'
3	'Y'
4	'Z'

$\div$

B	
common	
'X'	
'Y'	

$=$

A	
unique	
1	

2.4 Give all employees assigned to any department in which Bob is manager.

π name (
Employees $\bowtie$
 ρ d_name $\leftarrow$ name π name σ manager = 'Bob' Departments
)

```
SELECT DISTINCT E.name
FROM Employees E JOIN Departments D
ON E.d_name = D.name
WHERE D.manager = 'Bob' ;
```

Problem 3: Exam question (in 2017)

Consider the example data provided in the following tables:

- Relax: <https://dbis-uibk.github.io/relax/calc/gist/c42da7b7855e525e80e0a7255568e587>
- DB_fiddle: <https://www.db-fiddle.com/f/itidCoYCpBTsgf1jsRPEkd/4>

Assignment	
project_id	emp_id
1001	5
1001	7
1002	9
1002	2
1003	3
1004	3
1004	8
1005	1
1006	6

Department	
dept_id	name
1	Marketing
2	Finance
3	Human Resources
4	R&D
5	Management

Project	
project_id	name
1001	New Training
1002	Bus Stop Billboards
1003	Recruitment Drive
1004	Brain Drain
1005	The Ultimate Product
1006	Solo Sing Career

Employee				
emp_id	name	sex	salary	dept_id
1	Bob	M	26,000	4
2	Bill	M	49,000	5
3	Jim	M	28,000	3
4	Jill	F	48,000	5
5	Jane	F	35,000	1
6	Hank	M	35,000	4
7	Joan	F	37,000	2
8	Tom	M	39,000	4
9	Ron	M	29,000	1

Note that an employee can be assigned to multiple departments, and a salary is specific to the work done in that department.



Problem 3: Exam question (in 2017)

- 1) Give the names of all employees currently working on more than one project.
- 2) Give the name of the highest paid employee working on Brain Drain.
- 3) Give the names of all projects on which only men are working.
- 4) Give the names of all departments that currently have no employees involved in any projects.
- 5) Give the names of departments where all employees are active in a project.



3.1 Give the names of all employees currently working on more than one project.

```
π emp_id σ project_id2 < project_id (  
    Assignment ⋈  
    ρ project_id2 ← project_id Assignment  
)
```

```
SELECT DISTINCT A1.emp_id  
FROM Assignment A1 JOIN Assignment A2  
ON A1.emp_id = A2.emp_id  
WHERE A1.project_id <> A2.project_id ;
```

3.2 Give the name of the highest paid employee working on Brain Drain.

```
Emp_salary =  
π name, salary  
(  
    π emp_id (σ name = 'brain drain' Project ⋈ Assignment)  
    ⋈  
    Employee  
)  
  
π name Emp_salary  
-  
π name σ salary < salary2  
(  
    Emp_salary  
    ×  
    ρ name2 ← name, salary2 ← salary Emp_salary  
)
```

```
CREATE VIEW Emp_salary AS  
SELECT name, salary  
FROM Employee  
WHERE emp_id IN (  
    SELECT emp_id  
    FROM Assignment NATURAL JOIN Project  
    WHERE name = 'brain drain'  
);  
  
SELECT name  
FROM Emp_salary E  
WHERE NOT EXISTS (  
    SELECT *  
    FROM Emp_salary S  
    WHERE E.salary < S.salary  
);
```

3.3 Give the names of all projects on which only men are working.

```
π name
(
  Project -
  (
    Project ⋈ π project_id
    (
      σ sex ≠ 'M' Employee ⋈ Assignment
    )
  )
)
```

```
SELECT name
FROM Project P
WHERE NOT EXISTS
(
  SELECT *
  FROM Assignment NATURAL JOIN Employee
  WHERE sex <> 'M' AND project_id = P.project_id
);
```

3.4 Give the names of all departments that currently have no employees involved in any projects.

```
π name
(
  Department -
  (
    π dept_id (Employee ⋈ Assignment) ⋈ Department
  )
)
```

```
SELECT name
FROM Department D
WHERE NOT EXISTS
(
  SELECT *
  FROM Employee E NATURAL JOIN Assignment
  WHERE E.dept_id = D.dept_id
);
```

3.5 Give the names of departments where all employees are active in a project.

```
π name
(
  Department ⋈
  (
    π dept_id Department -
    π dept_id
    (
      π emp_id, dept_id Employee -
      π emp_id, dept_id ( Employee ⋈ Assignment)
    )
  )
)
```

```
SELECT name
FROM Department
WHERE dept_id NOT IN
(
  SELECT dept_id
  FROM Employee
  WHERE emp_id NOT IN
  (
    SELECT emp_id
    FROM Assignment
  )
);
```

